



C Programming:

Decision-Making and Iteration



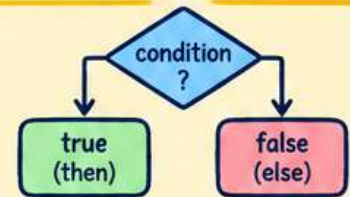
1

Decision-Making Constructs

Selection logic enables **selective execution** based on boolean conditions.

Constructs:

- if
- if-else
- nested if
- else-if ladder
- conditional operator (ternary ?:)
- switch



Logic:

- **if-else** handles complex ranges.
- **switch** is optimized for fixed choices in menu-driven programs.

2

Iterative Statements

Looping logic minimizes code redundancy by repeating code blocks.

```
for (i = 0; i < n; i++) {  
    for (j = 0; j < m; j++) {  
        // process  
    }  
}
```

Loop Types:

- **for** (fixed count)
- **while** (entry-controlled condition)
- **do-while** (exit-controlled)



Control Differences:

- **for** and **while** allow zero executions.
- **do-while** guarantees at least one execution.

Nested Loops:

- **Loops** inside loops are used for multi-level processing, such as matrices and patterns.

3

Control Flow Modifiers

These commands alter the standard **execution** path of loops and blocks.

Break:

Immediately terminates the execution of the current loop or switch block.



Continue:

Skips the remainder of the current iteration and jumps directly to the next cycle.



4

Algorithmic Patterns

Mathematical programming frequently combines loops and conditions to process data.

**Loops + Conditions
= Powerful Programs**

Digit Extraction:

Use `number % 10` to get the last digit and `number / 10` to remove it; repeat until `number == 0`.

```
number % 10  
number / 10  
while (number != 0)
```

Max/Min Search:

Initialize a variable with the first input, then iterate and update if a new value satisfies the comparison condition.



Pattern Logic:

Outer loops define row count while inner loops handle individual column output.



Introduction

The **if statement** is the simplest decision-making statement in C. It is used when a particular statement or group of statements needs to be executed **only when a given condition is true**. The condition is checked before executing the statements inside the **if** block. If the condition is true, the block is executed; if it is false, the block is skipped. It is useful for checking conditions such as eligibility, minimum marks, age requirements, balance availability, and other situations where an action is required only when a condition is satisfied.

Example

A college wants to display "**Eligible for exam**" when a student's attendance is 75% or above.

If the attendance is 80%, the message is displayed. If the attendance is 60%, the **if** block is skipped.

Syntax

```
if (condition)
{
    statement;
}
```

Program

```
#include <stdio.h>

int main()
{
    int attendance;

    printf("Enter attendance percentage: ");
    scanf("%d", &attendance);

    if (attendance >= 75)
    {
        printf("Student is eligible for examination.");
    }

    return 0;
}
```

Input

```
Enter attendance percentage: 82
```

Output

```
Student is eligible for examination.
```

1.2 if-else Statement

Introduction

The **if-else statement** is used when a program needs to choose between **two possible alternatives**. If the condition is true, the statements inside the **if** block are executed. If the condition is false, the statements inside the **else** block are executed. Therefore, it provides a separate action for both true and false conditions. It is commonly used for problems such as pass or fail, even or odd, eligible or not eligible, and positive or negative. The **if-else** statement makes it possible for the program to respond appropriately to either outcome.

Example

An ATM checks whether the customer's account has enough balance for withdrawal.

- If sufficient balance is available → Withdrawal is allowed.
- Otherwise → Withdrawal is rejected.

Syntax

```
if (condition)
{
    statement1;
}
else
{
    statement2;
}
```

Program

```
#include <stdio.h>

int main()
{
    int number;

    printf("Enter a number: ");
```

```
scanf("%d", &number);

if (number % 2 == 0)
{
    printf("The number is even.");
}
else
{
    printf("The number is odd.");
}

return 0;
}
```

Input

Enter a number: 24

Output

The number is even.

1.3 Nested if Statement

Introduction

A **nested if statement** is an **if** statement placed inside another **if** statement. It is used when a second condition needs to be checked only after the first condition has been satisfied. In this structure, the outer **if** performs the first level of checking and the inner **if** performs another level of checking. It is useful when decisions are dependent on one another. For example, a college may first check whether a student has passed and then check whether the student has obtained distinction marks. The inner **if** is considered only when the outer condition is true.

Example

A scholarship system performs two checks:

1. First, it checks whether the student's percentage is at least 75%.
2. If the student is eligible, it checks whether the percentage is at least 90%.
3. Students satisfying both conditions receive a special scholarship.

Syntax

```
if (condition1)
{
    if (condition2)
    {
        statement;
    }
}
```

Program

```
#include <stdio.h>

int main()
{
    int marks;

    printf("Enter marks: ");
    scanf("%d", &marks);

    if (marks >= 75)
    {
        if (marks >= 30)
        {
            printf("Eligible for special scholarship.");
        }
    }

    return 0;
}
```

Input

Enter marks: 34

Output

Eligible for special scholarship.

1.4 else-if Ladder

Introduction

The `else-if` **ladder** is used when a program needs to check **multiple conditions** and perform a different action for each condition. The conditions are checked one after another from top to bottom. As soon as a true condition is found, its corresponding block is executed and the remaining conditions are skipped. If none of the conditions is true, the final `else` block is executed, if it is present. It is commonly used for assigning grades, calculating discounts, classifying marks, determining tax slabs, and other problems involving multiple possible outcomes.

Example

A college assigns grades according to marks:

- 90 and above → A+
- 80–89 → A
- 70–79 → B
- 60–69 → C
- Below 60 → D

Since there are several possible outcomes, an `else-if` ladder is suitable.

Syntax

```
if (condition1)
{
    statement1;
}
else if (condition2)
{
    statement2;
}
else if (condition3)
{
    statement3;
}
else
{
    statement4;
}
```

Program

```
#include <stdio.h>

int main()
```



```
{  
    int marks;  
  
    printf("Enter marks: ");  
    scanf("%d", &marks);  
  
    if (marks >= 30)  
    {  
        printf("Grade A+");  
    }  
    else if (marks >= 80)  
    {  
        printf("Grade A");  
    }  
    else if (marks >= 70)  
    {  
        printf("Grade B");  
    }  
    else if (marks >= 60)  
    {  
        printf("Grade C");  
    }  
    else  
    {  
        printf("Grade D");  
    }  
  
    return 0;  
}
```

Input

Enter marks: 85

Output

Grade A

2. Conditional Operator (?:)

Introduction

The **conditional operator** is a special operator in C that provides a short way to make a decision between two alternatives. It is also called the **ternary operator** because it operates

on three operands. It first checks a condition and then selects one of two expressions depending on whether the condition is true or false. It is mainly used for simple two-way decisions where writing a complete `if-else` statement may be unnecessary. The conditional operator can also be used to assign a value to a variable based on a condition.

Example

An online shopping application provides free delivery when the order amount is ₹500 or more.

```
Order amount >= ₹500 → Free Delivery
Order amount < ₹500  → Delivery Charges
```

Syntax

```
condition ? expression1 : expression2;
```

If the condition is true, `expression1` is selected; otherwise, `expression2` is selected.

Program

```
#include <stdio.h>

int main()
{
    int marks;
    char *result;

    printf("Enter marks: ");
    scanf("%d", &marks);

    result = (marks >= 40) ? "Pass" : "Fail";

    printf("Result = %s", result);

    return 0;
}
```

Input

```
Enter marks: 72
```

Output

Result = Pass

3. `switch` Statement

Introduction

The `switch` **statement** is a multi-way decision-making statement in C. It is used when a program needs to select one action from several **fixed choices**. The value of an expression is compared with the values specified in different `case` labels. When a matching case is found, the statements associated with that case are executed. The `break` statement is normally used to terminate a case and prevent execution from continuing into the next case. A `default` case can be used when none of the specified cases matches. The `switch` statement is especially useful for creating menu-driven programs.

Example

A calculator provides four choices:

```
1 → Addition
2 → Subtraction
3 → Multiplication
4 → Division
```

The user's choice can be checked using a `switch` statement.

Syntax

```
switch (expression)
{
    case value1:
        statement1;
        break;

    case value2:
        statement2;
        break;

    case value3:
        statement3;
        break;

    default:
```

```
        statement4;  
    }
```

Program

```
#include <stdio.h>  
  
int main()  
{  
    int choice, a, b;  
  
    printf("1. Addition\n");  
    printf("2. Subtraction\n");  
    printf("3. Multiplication\n");  
    printf("4. Division\n");  
  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
  
    printf("Enter two numbers: ");  
    scanf("%d %d", &a, &b);  
  
    switch (choice)  
    {  
        case 1:  
            printf("Result = %d", a + b);  
            break;  
  
        case 2:  
            printf("Result = %d", a - b);  
            break;  
  
        case 3:  
            printf("Result = %d", a * b);  
            break;  
  
        case 4:  
            printf("Result = %d", a / b);  
            break;  
  
        default:  
            printf("Invalid choice.");  
    }  
  
    return 0;  
}
```

Input

```

1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice: 1
Enter two numbers: 20 10

```

Output

```
Result = 30
```

Difference Between `if-else` and `switch`

--		
1	It is used for decision-making based on conditions.	It is used for selecting one option from multiple fixed choices.
2	It can handle relational conditions such as <code>></code> , <code><</code> , <code>>=</code> , and <code><=</code> .	It mainly compares an expression with fixed <code>case</code> values.
3	It can handle logical operators such as <code>&&</code> , and <code>!</code> .	It does not directly support complex logical conditions in case labels.
4	It is suitable for checking ranges of values.	It is suitable for checking specific fixed values.
5	It can be used for conditions such as <code>marks >= 30</code> .	It can be used for choices such as <code>case 1</code> , <code>case 2</code> , etc.
6	Multiple conditions are written using <code>if</code> and <code>else-if</code> .	Multiple choices are written using <code>case</code> labels.
7	Conditions are evaluated one after another until a suitable condition is found.	The switch expression is compared with the available case values.
8	It is suitable for complex decision-making.	It is generally convenient for menu-driven programs.

No.	if-else	switch
9	It does not require a <code>break</code> statement to move between alternatives.	<code>break</code> is normally used to exit a matched case.
10	It provides greater flexibility for conditions and ranges.	It provides a cleaner structure when choices are based on fixed values.

B. ITERATIVE STATEMENTS

1. Iterative Statements

Introduction

In programming, some statements need to be executed **repeatedly** until a particular condition is satisfied. For example, if a teacher wants to display the roll numbers of 50 students, writing the same statement 50 times would be unnecessary. Similarly, an application may need to repeatedly accept user input, calculate values, or process records. **Iterative statements**, also called **looping statements**, are used to repeat a block of statements multiple times. They reduce code repetition, make programs shorter, and are very useful when performing repetitive tasks.

Example

Suppose a teacher wants to display the numbers from **1 to 10**.

Instead of writing:

```
Display 1
Display 2
Display 3
...
Display 10
```

a loop can repeat the same operation and display the numbers automatically.

Types of Iterative Statements

C provides three main iterative statements:

1. `for` loop
 2. `while` loop
 3. `do-while` loop
-

1.1 `for` Loop

Introduction

The `for` loop is an iterative statement used to execute a block of statements repeatedly. It is generally used when the number of repetitions is **known or can be determined in advance**. A `for` loop contains three important parts: initialization, condition, and update. The initialization is performed once at the beginning, the condition is checked before every iteration, and the update changes the loop variable after each iteration. It is commonly used for displaying numbers, generating multiplication tables, processing arrays, and performing calculations repeatedly.

Example

A teacher wants to display the roll numbers of students from **1 to 5**.

The program can use a `for` loop to automatically generate:

```
1
2
3
4
5
```

Syntax

```
for (initialization; condition; update)
{
    statement;
}
```

Program

```
#include <stdio.h>

int main()
{
    int i;
```

```
for (i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}

return 0;
}
```

Input

No input required.

Output

```
1
2
3
4
5
```

1.2 while Loop

Introduction

The **while loop** is used to repeatedly execute a block of statements as long as a specified condition remains true. It is an **entry-controlled loop** because the condition is checked before the loop body is executed. If the condition is false at the beginning, the loop body will not execute even once. The **while** loop is useful when the number of repetitions is not known in advance and depends on a condition. It is commonly used for input validation, searching, processing data, and situations where repetition continues until a particular condition becomes false.

Example

Consider a login system. The system may repeatedly ask the user to enter a password **while the password is incorrect**.

The number of attempts may not be known in advance, so a **while** loop can be used.

Syntax

```
while (condition)
{
    statement;
}
```

Program

```
#include <stdio.h>

int main()
{
    int i = 1;

    while (i <= 5)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

Input

No input required.

Output

```
1
2
3
4
5
```

1.3 do-while Loop

Introduction

The **do-while loop** is used to execute a block of statements repeatedly while a specified condition is true. It is an **exit-controlled loop** because the condition is checked after executing the loop body. Therefore, the statements inside the loop are executed **at least once**, even

when the condition is initially false. This makes the `do-while` loop useful when an operation must be performed at least once before checking whether it should continue. It is commonly used in menu-driven programs, input validation, and applications where the user must be given an option before deciding whether to continue.

Example

Consider an ATM menu:

1. Check Balance
2. Withdraw Money
3. Exit

The menu should be displayed at least once before asking the user whether to continue. Therefore, a `do-while` loop is suitable.

Syntax

```
do
{
    statement;
}
while (condition);
```

Note: A semicolon `;` is required after the `while(condition)`.

Program

```
#include <stdio.h>

int main()
{
    int i = 1;

    do
    {
        printf("%d\n", i);
        i++;
    }
    while (i <= 5);

    return 0;
}
```

Input

No input required.

Output

1
2
3
4
5

Difference Between for, while and do-while

--	-	-	Loop
1	It is an iterative statement used for repeated execution.	It is an iterative statement used for repeated execution.	It is an iterative statement used for repeated execution.
2	It is generally preferred when the number of iterations is known.	It is generally preferred when the number of iterations is not known in advance.	It is useful when the loop body must execute at least once.
3	Initialization, condition, and update are written together in the loop header.	Initialization is normally written before the loop.	Initialization is normally written before the loop.
4	The condition is checked before each iteration.	The condition is checked before each iteration.	The condition is checked after each iteration.
5	It is an entry-controlled loop.	It is an entry-controlled loop.	It is an exit-controlled loop.
6	The loop body may execute zero times if the initial condition is false.	The loop body may execute zero times if the initial condition is false.	The loop body executes at least once.
7	The update expression is generally written in the	The update is generally written inside the loop	The update is generally written inside the loop

No.	for Loop	while Loop	do-while Loop
	loop header.	body.	body.
8	It is commonly used for counting and fixed repetitions.	It is commonly used for condition-based repetition.	It is commonly used for menus and input validation.
9	Its syntax keeps loop control in one place.	Its syntax separates initialization and update from the condition.	Its condition appears at the end of the loop.
10	Example: displaying numbers from 1 to 100.	Example: reading values until a condition becomes false.	Example: displaying a menu at least once and then asking whether to continue.

2. Nested Loops

Introduction

A **nested loop** is a loop placed inside another loop. The outer loop controls the number of times the inner loop is executed. For every one iteration of the outer loop, the inner loop completes all its iterations. Nested loops are useful when a problem involves **repeated operations at more than one level**. They are commonly used for generating patterns, processing rows and columns of matrices, creating tables, and solving problems involving combinations of values. A nested loop can contain another `for`, `while`, or `do-while` loop.

Example

Consider a classroom with **3 rows and 4 students in each row**.

To process every student, we can use:

- Outer loop → controls the rows.
- Inner loop → controls the students in each row.

Thus, one loop operates inside another loop.

Syntax

```
for (initialization; condition; update)
{
    for (initialization; condition; update)
    {
        statement;
    }
}
```

Program

```
#include <stdio.h>

int main()
{
    int i, j;

    for (i = 1; i <= 3; i++)
    {
        for (j = 1; j <= 4; j++)
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}
```

Input

No input required.

Output

```
* * * *
* * * *
* * * *
```

3. break Statement

Introduction

The `break` **statement** is used to immediately terminate a loop or a `switch` statement. When the `break` statement is encountered inside a loop, the loop stops executing, and program control moves to the statement immediately following the loop. It is useful when the required result has already been obtained and continuing the loop is unnecessary. For example, while searching for a particular student record, the search can be stopped as soon as the record is found. The `break` statement helps avoid unnecessary iterations and provides better control over program execution.

Example

Suppose a program searches for the number **5** in a sequence of numbers.

Once the number 5 is found, there is no need to continue searching. The `break` statement can immediately terminate the loop.

Syntax

```
break;
```

Program

```
#include <stdio.h>

int main()
{
    int i;

    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
        {
            break;
        }

        printf("%d\n", i);
    }

    return 0;
}
```

Input

No input required.

Output

```
1
2
3
4
```

Here, when `i` becomes `5`, the `break` statement terminates the loop.

4. `continue` Statement

Introduction

The `continue` statement is used to skip the remaining statements of the current iteration and move to the **next iteration** of the loop. Unlike `break`, it does not terminate the entire loop. It is useful when a particular value or condition should be skipped while the remaining values continue to be processed. For example, a program may need to display numbers from 1 to 10 while skipping the number 5. In such a situation, `continue` can skip the current iteration and allow the loop to proceed with the next value.

Example

Suppose a program needs to display numbers from **1 to 5**, but number **3 should be skipped**.

The `continue` statement can skip the iteration when the value becomes 3.

Syntax

```
continue;
```

Program

```
#include <stdio.h>

int main()
{
    int i;

    for (i = 1; i <= 5; i++)
    {
        if (i == 3)
        {
```

```

        continue;
    }

    printf("%d\n", i);
}

return 0;
}

```

Input

No input required.

Output

```

1
2
4
5

```

Here, when `i` becomes `3`, `continue` skips that iteration and the loop proceeds with `4`.

Difference Between `break` and `continue`

1	It is used to terminate a loop or <code>switch</code> .	It is used to skip the current iteration of a loop.
2	It completely stops the execution of the loop.	It does not terminate the loop.
3	Control moves to the statement immediately after the loop or <code>switch</code> .	Control moves to the next iteration of the loop.
4	It can be used inside loops.	It is used inside loops.
5	It can also be used inside a <code>switch</code> statement.	It is not used to terminate a <code>switch</code> statement.

No.	break	continue
6	It is useful when no further iterations are required.	It is useful when the current iteration should be skipped.
7	The remaining iterations are not executed.	The remaining iterations are executed normally.
8	Example: stop searching when an item is found.	Example: skip invalid or unwanted values.
9	It provides complete exit from the loop.	It provides selective skipping within the loop.
10	After execution, the loop is permanently terminated.	After execution, the loop continues with the next iteration.

C. PROGRAMS USING DECISION-MAKING AND LOOPING

Introduction

Decision-making and looping are two important concepts used together in C programming. **Decision-making statements** help a program choose an action based on a condition, while **loops** allow a set of statements to be executed repeatedly. Many practical programming problems require both concepts at the same time. For example, a program may use a loop to process several numbers and an `if` statement to check whether each number is even, odd, positive, negative, prime, or divisible by another number. Combining decision-making and looping makes it possible to solve complex problems using fewer statements and less repetition.

Example

Suppose a teacher wants to identify the **even numbers between 1 and 10**.

The program needs to:

1. Generate numbers from 1 to 10 using a loop.
2. Check each number using a condition.

3. Display the number only if it is even.

Here, the **loop performs repetition**, while the **if statement performs the decision**.

1. Display Even Numbers from 1 to N

Introduction

An even number is a number that is completely divisible by 2. A program can use a loop to generate numbers from 1 to a specified limit and use an **if** statement to check whether each number is even. The modulus operator **%** gives the remainder after division. If a number divided by 2 gives a remainder of zero, it is an even number. This program demonstrates how a loop and decision-making statement can work together to solve a simple problem.

Example

If the user enters **10**, the program checks:

```
1 → Odd
2 → Even
3 → Odd
4 → Even
...
10 → Even
```

Only the even numbers are displayed.

Syntax

```
for (initialization; condition; update)
{
    if (condition)
    {
        statement;
    }
}
```

Program

```
#include <stdio.h>

int main()
{
```

```
int i, n;

printf("Enter the limit: ");
scanf("%d", &n);

for (i = 1; i <= n; i++)
{
    if (i % 2 == 0)
    {
        printf("%d ", i);
    }
}

return 0;
}
```

Input

Enter the limit: 10

Output

2 4 6 8 10

2. Display Odd Numbers from 1 to N

Introduction

An odd number is a number that is not completely divisible by 2. A loop can be used to generate numbers within a specified range, while an `if` statement can determine whether each number is odd. If the remainder obtained by dividing a number by 2 is not zero, the number is odd. This program is a simple example of combining repetition with decision-making. The same approach can be extended to many other number-based problems.

Example

If the limit is 10, the program displays:

1 3 5 7 9

Syntax

```
for (initialization; condition; update)
{
    if (number % 2 != 0)
    {
        statement;
    }
}
```

Program

```
#include <stdio.h>

int main()
{
    int i, n;

    printf("Enter the limit: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        if (i % 2 != 0)
        {
            printf("%d ", i);
        }
    }

    return 0;
}
```

Input

Enter the limit: 10

Output

1 3 5 7 3

3. Find the Largest of N Numbers

Introduction

Finding the largest number among several values is a common programming problem. A loop can be used to read the numbers one by one, while a decision-making statement can compare each number with the current largest value. Initially, the first number is considered the largest. Whenever a new number greater than the current largest is found, the largest value is updated. After all numbers are processed, the stored value represents the largest number. This program demonstrates how looping and decision-making can be combined for data processing.

Example

Suppose the numbers are:

```
25 48 12 67 34
```

The program compares the numbers one by one and finally identifies:

```
Largest = 67
```

Syntax

```
if (number > largest)
{
    largest = number;
}
```

Program

```
#include <stdio.h>

int main()
{
    int n, i, number, largest;

    printf("Enter the number of values: ");
    scanf("%d", &n);

    printf("Enter number 1: ");
    scanf("%d", &largest);

    for (i = 2; i <= n; i++)
    {
        printf("Enter number %d: ", i);
        scanf("%d", &number);

        if (number > largest)
```

```
        {
            largest = number;
        }
    }

    printf("Largest number = %d", largest);

    return 0;
}
```

Input

```
Enter the number of values: 5
Enter number 1: 25
Enter number 2: 48
Enter number 3: 12
Enter number 4: 67
Enter number 5: 34
```

Output

```
Largest number = 67
```

4. Find the Smallest of N Numbers

Introduction

The smallest number among a group of numbers can be found by comparing the values one by one. The first number is initially considered the smallest. A loop is then used to read the remaining numbers, and an `if` statement checks whether the newly entered number is smaller than the current smallest value. If it is smaller, the smallest value is updated. After all the numbers have been processed, the stored value represents the smallest number. This is another practical example of combining a loop with decision-making.

Example

For the numbers:

```
25 48 12 67 34
```

the smallest number is:

Syntax

```
if (number < smallest)
{
    smallest = number;
}
```

Program

```
#include <stdio.h>

int main()
{
    int n, i, number, smallest;

    printf("Enter the number of values: ");
    scanf("%d", &n);

    printf("Enter number 1: ");
    scanf("%d", &smallest);

    for (i = 2; i <= n; i++)
    {
        printf("Enter number %d: ", i);
        scanf("%d", &number);

        if (number < smallest)
        {
            smallest = number;
        }
    }

    printf("Smallest number = %d", smallest);

    return 0;
}
```

Input

```
Enter the number of values: 5
Enter number 1: 25
Enter number 2: 48
Enter number 3: 12
```

```
Enter number 4: 67
```

```
Enter number 5: 34
```

Output

```
Smallest number = 12
```

5. Calculate the Sum of N Numbers

Introduction

The sum of several numbers can be calculated by repeatedly adding each number to a variable. A loop is used to perform the addition for every number, while the program maintains a variable called `sum` to store the accumulated result. The value of `sum` is initially set to zero. During every iteration, the current number is added to `sum`. After the loop finishes, the final value of `sum` represents the total of all the entered numbers. This is one of the basic applications of looping in C.

Example

If the user enters:

```
10 20 30 40 50
```

the calculation is:

```
10 + 20 + 30 + 40 + 50 = 150
```

Syntax

```
sum = sum + number;
```

Program

```
#include <stdio.h>

int main()
{
    int n, i, number, sum = 0;
```

```
printf("Enter the number of values: ");
scanf("%d", &n);

for (i = 1; i <= n; i++)
{
    printf("Enter number %d: ", i);
    scanf("%d", &number);

    sum = sum + number;
}

printf("Sum = %d", sum);

return 0;
}
```

Input

```
Enter the number of values: 5
Enter number 1: 10
Enter number 2: 20
Enter number 3: 30
Enter number 4: 40
Enter number 5: 50
```

Output

```
Sum = 150
```

6. Multiplication Table

Introduction

A multiplication table is another simple application of looping. Instead of writing ten separate multiplication statements, a `for` loop can repeatedly multiply a given number by values from 1 to 10. During each iteration, the loop variable changes and produces the next line of the table. This demonstrates how repetitive calculations can be performed efficiently using a loop. Multiplication tables are also useful for understanding loop initialization, condition checking, updating, and repeated execution.

Example

For the number 5, the program generates:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

Syntax

```
for (i = 1; i <= 10; i++)
{
    result = number * i;
}
```

Program

```
#include <stdio.h>

int main()
{
    int number, i;

    printf("Enter a number: ");
    scanf("%d", &number);

    for (i = 1; i <= 10; i++)
    {
        printf("%d x %d = %d\n", number, i, number * i);
    }

    return 0;
}
```

Input

```
Enter a number: 5
```

Output

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
```

```
5 x 7 = 35
5 x 8 = 40
5 x 3 = 45
5 x 10 = 50
```

7. Count Positive, Negative and Zero Values

Introduction

Decision-making and looping can also be used to classify a set of values into different categories. A loop can read several numbers one by one, while `if-else-if` statements determine whether each number is positive, negative, or zero. Separate counter variables are used to keep track of the number of values in each category. Every time a value belongs to a particular category, its corresponding counter is increased. After all values have been processed, the program displays the count of positive, negative, and zero values.

Example

For the input:

```
10 -5 0 20 -8
```

the result is:

```
Positive = 2
Negative = 2
Zero = 1
```

Syntax

```
if (number > 0)
{
    positive++;
}
else if (number < 0)
{
    negative++;
}
else
{
    zero++;
}
```

Program

```
#include <stdio.h>

int main()
{
    int n, i, number;
    int positive = 0, negative = 0, zero = 0;

    printf("Enter the number of values: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        printf("Enter number %d: ", i);
        scanf("%d", &number);

        if (number > 0)
        {
            positive++;
        }
        else if (number < 0)
        {
            negative++;
        }
        else
        {
            zero++;
        }
    }

    printf("Positive numbers = %d\n", positive);
    printf("Negative numbers = %d\n", negative);
    printf("Zero values = %d", zero);

    return 0;
}
```

Input

```
Enter the number of values: 5
Enter number 1: 10
Enter number 2: -5
Enter number 3: 0
Enter number 4: 20
Enter number 5: -8
```

Output

Positive numbers = 2
Negative numbers = 2
Zero values = 1

D. NUMBER-BASED PROGRAMMING

Introduction

Number-based programming involves solving problems by performing calculations and applying logical operations on numbers. These programs help students understand how **decision-making statements and loops** can be used together to solve mathematical problems. Many basic programming problems involve checking the properties of a number, calculating a result, or generating a sequence of numbers. Number-based programs are also important because they strengthen logical thinking and provide practice with operators, conditions, loops, and variables. In this unit, the important number-based programs are **Prime Number, Factorial, Fibonacci Series, Armstrong Number, and Numeric Pattern Generation**.

Programs Covered

1. Prime Number
 2. Factorial
 3. Fibonacci Series
 4. Armstrong Number
 5. Numeric Pattern Generation
-

1. Prime Number

Introduction

A **prime number** is a positive integer greater than 1 that has exactly **two factors: 1 and the number itself**. For example, 2, 3, 5, 7, 11, and 13 are prime numbers. To determine whether a number is prime, the program checks whether it is divisible by any number other than 1 and itself. A loop can be used to perform these divisibility checks repeatedly. If the number is divisible by another number, it is not prime; otherwise, it is prime. Prime numbers are widely used in mathematics and also have important applications in computer science and cryptography.

Example

Consider the number 7.

The factors of 7 are:

1 and 7

Therefore, 7 is a prime number.

Consider 8:

Factors: 1, 2, 4, 8

Since it has factors other than 1 and itself, 8 is not prime.

Syntax

The basic logic used to check divisibility is:

```
if (number % i == 0)
{
    // number is divisible by i
}
```

Program

```
#include <stdio.h>

int main()
{
    int number, i, flag = 0;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (number <= 1)
    {
        printf("The number is not prime.");
    }
    else
    {
        for (i = 2; i < number; i++)
        {
            if (number % i == 0)
            {
                flag = 1;
            }
        }
    }
}
```

```
        break;
    }
}

if (flag == 0)
{
    printf("The number is prime.");
}
else
{
    printf("The number is not prime.");
}

return 0;
}
```

Input

Enter a number: 17

Output

The number is prime.

2. Factorial

Introduction

The **factorial** of a positive integer is the product of all positive integers from 1 up to that number. It is represented using the symbol $!$. For example, the factorial of 5 is written as $5!$ and is calculated as $5 \times 4 \times 3 \times 2 \times 1 = 120$. Factorial calculations are commonly used in mathematics, permutations, combinations, probability, and various programming problems. A loop is useful for calculating factorial because the multiplication operation needs to be repeated for each number. The factorial of zero is defined as 1.

Example

For 5:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

= 120

Syntax

```
factorial = factorial * i;
```

Program

```
#include <stdio.h>

int main()
{
    int n, i;
    long long factorial = 1;

    printf("Enter a number: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        factorial = factorial * i;
    }

    printf("Factorial = %lld", factorial);

    return 0;
}
```

Input

Enter a number: 5

Output

Factorial = 120

3. Fibonacci Series

Introduction

The **Fibonacci series** is a sequence in which each number is obtained by adding the previous two numbers. A commonly used Fibonacci sequence starts with **0 and 1**. The next number is calculated by adding these two values, and the process continues repeatedly. For example, the sequence is 0, 1, 1, 2, 3, 5, 8, 13. A loop can be used to generate the required number of terms. Fibonacci numbers are used in mathematical problems and can also be used to demonstrate sequence generation and repeated calculations in programming.

Example

The first 8 terms of the Fibonacci series are:

```
0 1 1 2 3 5 8 13
```

The calculation is:

```
0 + 1 = 1
1 + 1 = 2
1 + 2 = 3
2 + 3 = 5
3 + 5 = 8
```

Syntax

```
next = first + second;
first = second;
second = next;
```

Program

```
#include <stdio.h>

int main()
{
    int n, i;
    int first = 0, second = 1, next;

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        printf("%d ", first);

        next = first + second;
        first = second;
```



```
        second = next;  
    }  
  
    return 0;  
}
```

Input

Enter the number of terms: 8

Output

0 1 1 2 3 5 8 13

4. Armstrong Number

Introduction

An **Armstrong number** is a number that is equal to the sum of its digits, where each digit is raised to the power of the total number of digits. For example, 153 is an Armstrong number because it contains three digits and $1^3 + 5^3 + 3^3 = 153$. To check an Armstrong number, the program first separates the individual digits using the modulus and division operators. Each digit is then raised to the required power and added to a sum. Finally, the calculated sum is compared with the original number. If both values are equal, the number is an Armstrong number.

Example

Consider 153.

$$\begin{aligned} &1^3 + 5^3 + 3^3 \\ &= 1 + 125 + 27 \\ &= 153 \end{aligned}$$

Therefore:

153 is an Armstrong number.

Syntax

The basic digit extraction operation is:

```
digit = number % 10;
number = number / 10;
```

Program

```
#include <stdio.h>
#include <math.h>

int main()
{
    int number, original, digit, digits = 0;
    int sum = 0;

    printf("Enter a number: ");
    scanf("%d", &number);

    original = number;

    while (number != 0)
    {
        digits++;
        number = number / 10;
    }

    number = original;

    while (number != 0)
    {
        digit = number % 10;
        sum = sum + (int)pow(digit, digits);
        number = number / 10;
    }

    if (sum == original)
    {
        printf("The number is an Armstrong number.");
    }
    else
    {
        printf("The number is not an Armstrong number.");
    }

    return 0;
}
```

Input

Enter a number: 153

Output

The number is an Armstrong number.

5. Numeric Pattern Generation

Introduction

Numeric pattern generation is a programming technique used to display numbers in a particular arrangement of rows and columns. It mainly uses **nested loops**, where the outer loop controls the rows and the inner loop controls the numbers printed in each row. Different conditions and loop structures can be used to create different patterns. Pattern programs are useful for developing logical thinking and understanding how nested loops work. They are also common programming exercises because they require students to carefully understand the relationship between rows, columns, and loop variables.

Example

A simple numeric pattern with 5 rows can be displayed as:

```
1
12
123
1234
12345
```

Here:

- The outer loop controls the number of rows.
- The inner loop controls the numbers printed in each row.

Syntax

```
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= i; j++)
    {
        printf("%d", j);
    }
}
```

```
printf("\n");  
}
```

Program

```
#include <stdio.h>  
  
int main()  
{  
    int i, j, n;  
  
    printf("Enter the number of rows: ");  
    scanf("%d", &n);  
  
    for (i = 1; i <= n; i++)  
    {  
        for (j = 1; j <= i; j++)  
        {  
            printf("%d", j);  
        }  
  
        printf("\n");  
    }  
  
    return 0;  
}
```

Input

Enter the number of rows: 5

Output

```
1  
12  
123  
1234  
12345
```

UNIT 2 – IMPORTANT QUESTIONS

2 Marks Questions

1. Define decision-making statement in C.
2. Write the syntax of if-else statement.
3. What is a nested if statement?
4. What is a conditional operator? Write its symbol.
5. What is a switch statement?
6. What is looping? Name the different types of loops in C.
7. What is an entry-control loop? Give one example.
8. What is an exit-control loop? Give one example.
9. What is the purpose of the break statement?
10. What is the purpose of the continue statement?

5 Marks Questions

1. Explain the if statement with syntax and suitable example.
2. Write the syntax and flowchart for if-else statement with an example.
3. Explain nested if statement with syntax and example.
4. Explain else-if ladder with syntax and suitable example.
5. Explain the switch statement with syntax and example.
6. Explain for loop with syntax and suitable example.
7. Explain while loop with syntax and suitable example.
8. Explain do-while loop with syntax and suitable example.
9. Differentiate between while and do-while loops.
10. Write a C program to find the largest of three numbers.

10 Marks Questions

1. Explain the different decision-making statements in C with syntax, flowchart and suitable examples.
2. Explain the switch statement with syntax, flowchart and suitable example. Explain the use of case, break and default.
3. What is looping? Explain for, while and do-while loops with syntax, flowchart and suitable examples.
4. Explain nested loops, break and continue statements with syntax and suitable examples.
5. Explain number-based programming with suitable C programs for Prime number, Factorial, Fibonacci series and Armstrong number.